

Lab Experiment – 02

8-Queen Problem

Aim

To solve the 8-Queen problem using Backtracking Algorithm.

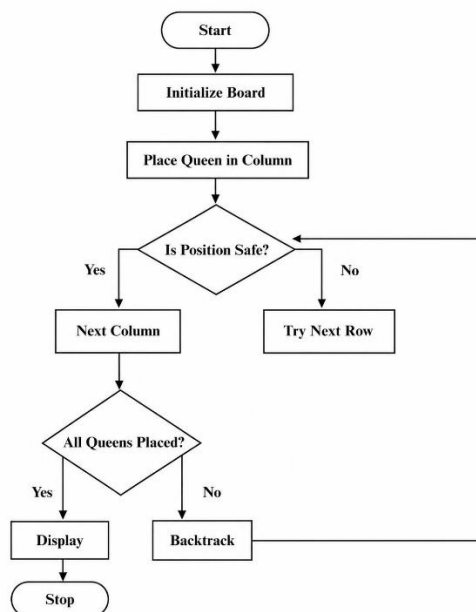
Objective

To place eight queens on an 8×8 chessboard so that no two queens attack each other.

Algorithm

1. Start with an empty chessboard.
2. Place a queen in the first column.
3. Check if the position is safe.
4. If safe, place the queen and move to the next column.
5. If no safe position exists, backtrack to the previous column.
6. Repeat until all eight queens are placed.
7. Display the final board configuration.

Flowchart



CODE:

N = 8

```
def print_board(board):
```

```
    for row in board:
```

```
        print(" ".join("Q" if x else "." for x in row))
```

```
def is_safe(board, row, col):
```

```
    for i in range(col):
```

```
        if board[row][i]:
```

```
            return False
```

```
    i, j = row, col
```

```
    while i >= 0 and j >= 0:
```

```
        if board[i][j]:
```

```
            return False
```

```
        i -= 1
```

```
        j -= 1
```

```
    i, j = row, col
```

```
    while i < N and j >= 0:
```

```
        if board[i][j]:
```

```
            return False
```

```
        i += 1
```

```
        j -= 1
```

```
    return True
```

```
def solve(board, col):
```

```
    if col >= N:
```

```
        return True
```

```
for i in range(N):
```

```
    if is_safe(board, i, col):
```

```
        board[i][col] = 1
```

```
        if solve(board, col + 1):
```

```
            return True
```

```
        board[i][col] = 0
```

```
    return False
```

```
board = [[0]*N for _ in range(N)]
```

```
if solve(board, 0):
```

```
    print("Solution:")
```

```
    print_board(board)
```

```
else:
```

```
    print("No Solution Exists")
```

OUTPUT:

```
C:\> Users > Asus > OneDrive > Desktop > Python Codes > Experiment 2.py > ...
1  N = 8
2
3  def print_board(board):
4      for row in board:
5          print(" ".join("Q" if x else "." for x in row))
6
7  def is_safe(board, row, col):
8      for i in range(col):
9          if board[row][i]:
10             return False
11
12     i, j = row, col
13     while i >= 0 and j >= 0:
14         if board[i][j]:
15             return False
16         i -= 1
17         j -= 1
18
19     i, j = row, col
20     while i < N and j >= 0:
21         if board[i][j]:
22             return False
23         i += 1
24         j -= 1
25
26     return True

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Q . . . . .
. . . . . Q .
. . . . Q . .
. . . . . Q
. Q . . . . .
. . . Q . . .
. . . . Q . .
. . Q . . . .
```

Result

The 8-Queen problem is solved successfully using the Backtracking algorithm.

Conclusion

The Backtracking algorithm efficiently places all eight queens on the chessboard such that no two queens attack each other.